

MovEase – Buy & Move Simplified

A Two-Sided Marketplace for House Buying & Moving Services

Table of Contents

1. Abstract
 2. Introduction
 3. Problem Statement
 4. Objectives
 5. Scope
 6. System Architecture
 7. Methodology
 8. Technologies Used
 9. Implementation Details
 - ❖ Authentication & Role Management
 - ❖ Mover Dashboard & Plan Generation
 - ❖ Service Provider Dashboard
 - ❖ Booking & Rating Lifecycle
 - ❖ Map Integration
 - ❖ Push Notifications
 - ❖ UI/UX Design
 10. Testing
 11. Screenshots
 12. Limitations
 13. Future Work
 14. Conclusion
 15. References
-

1. Abstract

MovEase is an Android application that streamlines the process of relocating to a new home in Lahore. It serves as a two-sided marketplace connecting **movers** (people looking to buy a house and move) with **service providers** (labour, packing material, transport). The app automatically generates optimised moving plans by combining real estate listings with available services, ranks them by cost and quality, and highlights advantages and disadvantages. Users can book a plan, track its status, and rate services after completion. The system uses Firebase for authentication and real-time data, a custom combination engine, and a material design interface for an intuitive experience.

2. Introduction

Moving to a new house is a complex, multi-vendor task. People must search for property, hire labour, buy packing supplies, and arrange transport – often from separate sources. MovEase solves this by **unifying all these needs into a single app**. The platform not only helps users find a house but also generates complete moving plans, giving them a clear, ranked list of options with transparent pros and cons. Service providers can register, list their offerings, and receive orders directly from movers, creating a dynamic, two-sided marketplace.

3. Problem Statement

Current solutions for home relocation in Lahore are fragmented. Users must:

- Search multiple real-estate websites.
- Contact various labour contractors, packing suppliers, and transport companies individually.
- Manually compare costs and reliability.
- Manage bookings and service quality without a central platform.

There is no integrated system that offers a **single-click moving plan** combining house purchase/rental with all required services, complete with ratings, real-time updates, and booking management.

4. Objectives

- Build an Android app that generates personalised moving plans using real property data and user-provided services.
 - Implement a **two-sided marketplace** where movers and service providers can interact.
 - Provide a scoring system that considers both **cost and user ratings** to rank plans.
 - Enable a full **booking lifecycle** (book → accept → complete → rate).
 - Offer a **free map view** of house locations.
 - Ensure the UI is accessible, modern, and follows Material Design guidelines.
-

5. Scope

- **Target audience:** Residents of Lahore, Pakistan.
 - **Platform:** Android (minSdk 24).
 - **Services covered:** Labour, packing material, transport.
 - **Property data:** 20 recent real-estate listings scraped from [Zameen.com](https://www.zameen.com), hardcoded as mock API responses.
 - **Authentication:** Firebase Email/Password.
 - **Database:** Cloud Firestore.
-

6. System Architecture

The app follows a **client-server architecture** with a mock API layer.

- **Client:** Android app (Java, XML).
- **Mock API:** OkHttp interceptor returns hardcoded JSON for houses; services are fetched from Firestore.
- **Authentication:** Firebase Auth.
- **Data storage:** Firestore (users, services, bookings).
- **Notifications:** Firebase Cloud Messaging.

Data Flow:

text

Mover Input → Repository → (Mock Houses + Firestore Services) → PlanGenerator → Ranked Plans → UI

7. Methodology

The project was developed using an iterative approach:

1. Requirements gathering and system design.
 2. Setting up Firebase and authentication with role management.
 3. Implementing the moving-plan input form and the mock API layer.
 4. Building the **PlanGenerator** engine (Cartesian product, scoring, pros/cons).
 5. Creating the provider dashboard (add/edit/delete services) and linking to Firestore.
 6. Adding the booking workflow (create booking, accept, complete, rate).
 7. Integrating map, push notifications, and UI theming.
 8. Testing on a real device and emulator.
-

8. Technologies Used

Component	Technology
Frontend	Java, XML, Material Components, RecyclerView, CardView
Backend	Firebase (Auth, Firestore, Cloud Messaging)
Networking	Retrofit2, OkHttp3 (with custom interceptor)
Map	WebView + Leaflet.js + OpenStreetMap (free)
Build Tools	Gradle (Groovy DSL)

Component	Technology
Testing	Manual testing on real device/emulator

9. Implementation Details

9.1 Authentication & Role Management

- Users sign up with email/password.
- A role is selected (Mover is the default).
- The role is stored in Firestore under a `users` collection.
- On login, the app fetches the role and routes to the appropriate dashboard.
- Session is managed by Firebase Auth; no custom token handling required.

9.2 Mover Dashboard & Plan Generation

- The mover fills a form: desired area, max budget, bedrooms, moving date, and checks which services to include.
- If the user already has a house, they can check a box and enter a destination address instead.
- The **PlanGenerator** takes all available houses (mock) and services (Firestore), generates every possible combination, filters by budget, and scores each plan:

$$\text{Score} = 0.6 \times \text{costScore} + 0.4 \times \text{ratingScore}$$
- Top 5 plans are displayed with **advantages** (e.g., "Highly rated house", "Top-rated labour") and **disadvantages** (e.g., "No parking", "Labour rated below 3.5").
- Scores are shown out of 5 for user-friendliness.

9.3 Service Provider Dashboard

- Providers can add, edit, and delete services (labour, packing, transport).
- Each new service gets an **automatic initial rating** based on its cost (cheaper = higher rating, up to 5.0).
- Providers see a "My Orders" screen listing bookings that contain their services, with options to **Accept** and **Complete** each order.

9.4 Booking & Rating Lifecycle

- When a mover books a plan, a document is created in the `bookings` collection with status `pending`.
- The provider accepts → status `accepted`.
- The provider marks as completed → status `completed`.
- The mover sees completed bookings in “My Bookings” and can rate each included service (1-5 stars).
- The average rating of each service is recalculated and stored back to Firestore.

9.5 Map Integration

- Tapping “Map” on a plan card opens a WebView that loads an HTML page using **Leaflet.js** and **OpenStreetMap** tiles.
- A marker is placed at the house’s coordinates.
- No Google Maps API key is required, making it completely free.

9.6 Push Notifications

- Firebase Cloud Messaging is configured with a service that extends `FirebaseMessagingService`.
- Test notifications can be sent from the Firebase Console and appear on the device.

9.7 UI/UX Design

- **Theme:** Slate Gray primary, Warm Cream background, high-contrast text.
- All icons are vector drawables with content descriptions.
- Touch targets are at least 48dp.
- The logout button is filled red with an “X” icon for immediate recognition.
- Toolbars include back navigation, and pages are scrollable where needed.

10. Testing

- **Unit tests:** The PlanGenerator logic was tested with sample data to verify correct combination generation and scoring.
- **Integration tests:** The mock API interceptor was verified to return correct JSON.

- **Manual testing:**

- Signup and login with role routing.
- Mover form validation (empty fields, huge/small budgets).
- Plan generation with different service checkboxes.
- Booking flow (book, accept, complete, rate).
- Map display with real coordinates.
- Push notification receipt.

Known Issues Resolved:

- Resource linking errors due to missing drawables → added all required vector icons.
 - Serializable crash → wrapped sub-list in new ArrayList.
 - "No plans found" with huge budget → fixed hardcoded JSON data and repository threading.
 - Provider orders not appearing → corrected service ID assignment in MoveRepository.
-

11. Screenshots

Splash Screen

Login

Plan Input

Plan List

Plan Detail

Map

Mover Bookings

Rate Services

Provider My Orders

12. Limitations

- The property data is hardcoded and not fetched in real-time.
- The map coordinates for some houses are set to 0,0 (fallback to central Lahore).
- Push notifications are only tested with the Firebase Console; no custom server integration.

- No payment gateway integration.
 - The app is designed for a single city (Lahore) and a small dataset.
-

13. Future Work

- Integrate a real backend API to fetch live property listings.
 - Add **payment processing** to complete transactions.
 - Implement **real-time chat** between movers and providers.
 - Expand to other cities and property types (rentals, commercial).
 - Add **push notification automation** for status changes.
 - Improve the map with real coordinates for all properties.
 - Publish on Google Play Store.
-

14. Conclusion

MovEase successfully demonstrates a **complete two-sided marketplace** that simplifies the moving process. It combines real-world data, algorithmic planning, and a modern UI to deliver a unique, user-centric solution. The project showcases proficiency in Android development, Firebase integration, and system design, and fulfills all the requirements of a semester project with a working, testable APK.

15. References

- Firebase Documentation – <https://firebase.google.com/docs>
- Retrofit – <https://square.github.io/retrofit/>
- OpenStreetMap – <https://www.openstreetmap.org/>
- Leaflet.js – <https://leafletjs.com/>
- [Zameen.com](https://zameen.com) – (data source, accessed June 2026)

Made by: Muhammad Abdullah Tariq & Hannan Shoaib